

Fiches projets — 3 projets vitrines

Assistant RAG multi-documents · Agent d'extraction de documents · Intégration IA dans une appli existante

Chaque fiche suit la même structure, pensée pour cadrer un projet vitrine avant de s'y mettre :

- **Preliminaires** — le contexte, la stack conseillée et le budget estimé, à valider avant de démarrer.
- **Must** — le socle obligatoire : sans ça, le projet ne remplit pas son rôle de démo.
- **Should** — ce qui distingue un POC d'un produit fini et qui justifie le temps investi.
- **Could** — des bonus pour marquer les esprits si tu as le temps, sans être indispensables.

Méthode : priorité stricte Must > Should > Could. En cas de manque de temps, un projet avec tous les Must et aucun Could reste vendable ; l'inverse ne l'est pas.

Assistant RAG multi-documents avec citations

« Vos employés ou vos clients trouvent l'info en 5 secondes au lieu de 10 minutes. »

DURÉE ESTIMÉE 1,5 à 2 semaines	BUDGET DEV (API/INFRA) ~5 à 20 €/mois	STACK CONSEILLÉE Angular/Vue + API LLM + Vector DB	FAMILLE DE BESOIN Chercher dans de la donnée
-----------------------------------	--	--	--

C'est la mission la plus fréquente en freelance IA : toute entreprise avec de la documentation interne, un catalogue produit ou un support client veut ce type d'outil. Le ROI se voit immédiatement et se vend sans jargon technique — ce qui en fait le meilleur projet pour ouvrir une conversation commerciale.

PRÉLIMINAIRES *à cadrer avant de démarrer*

- **Objectif du projet** — démontrer une maîtrise complète du pipeline RAG (retrieval-augmented generation), de l'ingestion à la réponse citée, avec une UX de niveau produit.
- **Prérequis techniques** — un compte API LLM (Anthropic, OpenAI ou Mistral), un vector store (Qdrant Cloud, Pinecone ou Supabase — tiers gratuit suffisant), un backend (Node/Python) et un frontend (Angular ou Vue, ton point fort).
- **Jeu de données de démo** — prépare 3 à 5 documents réalistes et variés (PDF, FAQ, doc interne) pour que la démo ait l'air d'un vrai cas d'usage plutôt que d'un test bidon.
- **Budget à surveiller** — mets une alerte de budget côté provider dès le départ ; un RAG mal optimisé (tout le corpus envoyé en contexte) peut faire exploser la facture inutilement.

MUST — Fondations obligatoires *le MVP, non négociable*

- **Ingestion multi-documents** — upload et parsing d'au moins un format (PDF) avec découpage en chunks cohérents.
- **Recherche sémantique** — embeddings + stockage vectoriel + retrieval fonctionnel sur plusieurs documents à la fois.
- **Génération de réponse citée** — la réponse du LLM s'appuie sur les passages récupérés et cite la source exacte (nom du document, page ou section).
- **Interface chat fonctionnelle** — un utilisateur peut poser une question et recevoir une réponse lisible, sans bug bloquant.

SHOULD — Ce qui fait la différence *ce qui transforme le POC en produit*

- **Streaming token par token** — la réponse s'affiche progressivement comme ChatGPT, pas d'un bloc après un long chargement.
- **Historique multi-tours** — le bot garde le contexte de la conversation sur plusieurs échanges (mémoire courte).
- **Citations cliquables** — cliquer sur la source ouvre ou surligne le passage exact dans le document d'origine.
- **Interface soignée** — un vrai souci d'UX (états de chargement, erreurs propres, responsive) — c'est ce qui distingue un POC d'un produit.

COULD — Bonus pour impressionner *si le temps le permet*

- **Multi-formats** — supporter aussi .docx, .txt ou des pages web en plus du PDF.
- **Espaces documentaires multiples** — permettre de créer plusieurs bases (« projet client A », « projet client B ») dans la même appli.
- **Feedback utilisateur** — boutons pouce haut/bas sur chaque réponse pour simuler une boucle d'amélioration continue.
- **Comparateur de modèles** — basculer entre deux LLM pour montrer une compréhension fine des compromis coût/qualité.

Agent d'extraction et structuration de documents

« Ce que vos équipes ressaisissent à la main en une heure, l'agent le fait en quelques secondes. »

DURÉE ESTIMÉE ~1,5 semaine	BUDGET DEV (API/INFRA) ~5 à 15 €/mois	STACK CONSEILLÉE API LLM (structured output) + frontend de validation	FAMILLE DE BESOIN Automatiser une tâche répétitive
-------------------------------	--	--	---

C'est la tâche la plus citée par les PME quand elles pensent « automatisation IA ». Le gain de temps se chiffre facilement en euros, ce qui en fait l'argument commercial le plus simple à formuler face à un prospect qui ne connaît rien à l'IA.

PRÉLIMINAIRES *à cadrer avant de démarrer*

- **Objectif du projet** — montrer une extraction de données fiable via LLM (pas de l'OCR + règles rigides), avec une interface qui assume que l'IA se trompe parfois.
- **Prérequis techniques** — API LLM avec sortie structurée (function calling / JSON mode), et un fallback OCR (pytesseract) si les PDF sont scannés plutôt que natifs.
- **Jeu de données de démo** — rassemble un petit lot de faux documents variés (factures, devis, CV) pour prouver que le système généralise, pas qu'il a été calé sur un seul exemple.
- **Point d'attention produit** — penser dès le départ « qui valide les champs extraits avant l'export » — c'est ce détail qui montre une compréhension de l'usage réel en entreprise.

MUST — Fondations obligatoires *le MVP, non négociable*

- **Upload en lot** — possibilité de charger plusieurs PDF en une seule fois.
- **Extraction via LLM** — les champs clés (dates, montants, noms, références) sont extraits automatiquement, pas via des règles regex figées.
- **Export structuré** — export CSV propre et exploitable des données extraites.

SHOULD — Ce qui fait la différence *ce qui transforme le POC en produit*

- **Interface de vérification** — un écran de relecture/correction manuelle des champs avant l'export final.
- **Multi-formats de documents** — gérer factures, devis et CV avec un schéma de champs qui s'adapte au type de document.
- **Export Excel** — export .xlsx via l'API en plus du CSV, avec mise en forme minimale.
- **Indicateur de confiance** — signaler visuellement les champs extraits avec une confiance faible, pour guider la relecture.

COULD — Bonus pour impressionner *si le temps le permet*

- **Détection automatique du type** — le système reconnaît seul s'il traite une facture, un devis ou un CV, sans que l'utilisateur le précise.
- **Apprentissage des corrections** — les corrections manuelles enrichissent les prochains prompts (few-shot dynamique) pour s'améliorer avec l'usage.
- **Intégration sortante** — webhook ou export automatique vers un outil tiers (compta, CRM, Google Sheets).
- **Dashboard de suivi** — vue d'ensemble des lots traités, taux d'erreur estimé, temps gagné cumulé.

Intégration IA dans une appli existante (recherche / recommandation)

« La plupart des boîtes n'ont rien à construire de zéro — elles veulent ajouter de l'IA à ce qui existe déjà. »

DURÉE ESTIMÉE 2 à 3 semaines	BUDGET DEV (API/INFRA) ~5 à 15 €/mois	STACK CONSEILLÉE Appli existante + recherche sémantique / embeddings	FAMILLE DE BESOIN Améliorer un produit existant
---------------------------------	--	---	---

C'est la démo la plus forte compte tenu de ton profil full-stack : elle prouve que tu sais intégrer proprement l'IA dans une architecture existante, pas juste prototyper un script isolé. C'est exactement le type de mission le plus fréquent en freelance.

PRÉLIMINAIRES *à cadrer avant de démarrer*

- **Objectif du projet** — repartir d'un projet perso solide (ou construire une petite appli e-commerce/catalogue simple) et y greffer une brique IA intégrée proprement.
- **Prérequis techniques** — une base de données produits/contenus existante, une API LLM ou un modèle d'embeddings, et un backend capable d'exposer les deux modes de recherche.
- **Choix de base** — si tu réutilises un projet perso déjà solide, la durée peut descendre à ~1 semaine — sinon prévois le temps de construire une base catalogue crédible.
- **Angle de présentation** — prépare dès maintenant le comparatif avant/après (recherche mots-clés vs recherche intelligente), c'est l'élément qui convainc le plus vite un prospect non technique.

MUST — Fondations obligatoires *le MVP, non négociable*

- **Application de base fonctionnelle** — un catalogue ou une appli e-commerce simple mais réel, pas une maquette statique.
- **Recherche sémantique intégrée** — une recherche par sens (embeddings) vient compléter ou remplacer la recherche par mots-clés classique.
- **Démonstration avant/après** — l'utilisateur peut comparer visuellement le résultat de la recherche classique et celui de la recherche IA sur la même requête.

SHOULD — Ce qui fait la différence *ce qui transforme le POC en produit*

- **Système de recommandation** — un module « produits similaires » ou « vous pourriez aimer » basé sur les embeddings, pas sur des règles statiques.
- **Architecture propre** — l'IA est une brique du produit (service dédié, appelé proprement), pas un script isolé collé à l'interface.
- **UX comparative soignée** — les deux modes de recherche sont présentés côte à côte de façon claire, pas juste dans les logs ou la console.

COULD — Bonus pour impressionner *si le temps le permet*

- **Personnalisation** — les recommandations s'ajustent à l'historique de navigation ou d'achat simulé de l'utilisateur.
- **A/B testing chiffré** — mesurer et afficher un indicateur de pertinence (ex. temps jusqu'au bon résultat) entre les deux modes de recherche.
- **Recherche hybride** — combiner recherche sémantique et filtres classiques (prix, catégorie) dans une seule expérience fluide.
- **Analytics de pertinence** — un mini dashboard qui suit les requêtes sans résultat satisfaisant, pour montrer une logique d'amélioration continue.